Lab 2: Types of Data

CPE232 Data Models

WISIT SUWANNAO 67070501042

[1] CSV

[1]:
```
import csv
```

1.1 Writing new csv file

Note: Remember this example? We've already seen it in the last lab.

[2]:
```
with open('test.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['Name', 'Surname'])
    writer.writerow(['Alice', 'Johnson'])
    writer.writerow(['Bob', 'Smith'])
```

1.2 Reading a csv file

[3]:
```
with open('test.csv', 'r') as file:
    reader = csv.reader(file)
    header = next(reader)
    print(header)

    for row in reader:
        print(row)
```

[3]:
```
['Name', 'Surname']
['Alice', 'Johnson']
['Bob', 'Smith']
```

1.3 Use pandas to read csv file

Pandas is an open-source Python library used for data manipulation and analysis.

think of it as **Excel for Python**. It allows you to work with structured data (like spreadsheets or SQL tables) programmatically and efficiently. https://www.w3schools.com/python/pandas/default.asp

[4]:
```
import pandas as pd

df = pd.read_csv('test.csv')

df
```

[4]:

	Name	Surname
0	Alice	Johnson
1	Bob	Smith

(5) Write a Python script that reads the **students.csv** file and prints the content of the first 10 students row by row.

[5]:
```
import csv

file_path = 'students.csv'

with open(file_path, 'r', newline='') as file:
    reader = csv.reader(file)
    header = next(reader) # Read the header row
    print("Header: {header}")

    print("First 10 student rows:")
    for i, row in enumerate(reader):
        if i < 10:
            print(row)
        else:
            break
```

[5]:
```
Header: ['Name', 'Age', 'Grade']
First 10 student rows:
['Alice', '21', 'A']
['Bob', '22', 'B']
['Charlie', '20', 'C']
['David', '23', 'A']
['Eva', '21', 'B']
['Frank', '22', 'C']
['Grace', '20', 'A']
['Henry', '23', 'B']
['Ivy', '21', 'C']
['Jack', '22', 'A']
```

(6) List used the **students.csv** file into a pandas DataFrame. Use pandas to filter the DataFrame and create a new DataFrame containing only students who received an "A" grade. Print the new DataFrame.

[6]:
```
import pandas as pd

file_path = 'students.csv'
df_students = pd.read_csv(file_path)

df_a_grade = df_students[df_students['Grade'] == 'A']

print("Students with 'A' grade:")
print(df_a_grade)
```

[6]:
```
Students with 'A' grade:
    Name  Age  Grade
0  Alice   21      A
5  David   23      A
8    Eva   21      A
9  Frank   22      A
10 Grace   20      A
11  Henry   23      A
12    Ivy   21      A
13   Jack   22      A
14  Kelly   20      A
15  Liam   23      A
16    Mia   24      A
17  Nolan   20      A
18 Olivia   23      A
19  Paul   24      A
20  Quin   20      A
21  Ruby   24      A
22  Sara   23      A
23  Tina   20      A
24  Uma   24      A
25 Victor   23      A
26  Wendy   23      A
27  Xavier   24      A
28   Yara   20      A
29  Zoe   24      A
30  Adam   20      A
31 Brian   23      A
32  Chen   24      A
33  Diana   20      A
34  Elena   23      A
35  Fiona   24      A
36  Gavin   20      A
37   Hal   23      A
38   Ian   24      A
39  Jack   20      A
40  Kyle   24      A
41  Luna   23      A
42  Mason   20      A
43  Nina   23      A
44  Omar   24      A
45  Piper   20      A
46  Quinn   23      A
47  Rosa   24      A
48  Steve   20      A
49  Tara   23      A
50 Ulysses   24      A
51  Vera   20      A
52  Will   23      A
53  Xenia   24      A
54  Yuri   20      A
55  Zac   23      A
56  Alice   21      A
57  David   23      A
58    Eva   21      A
59  Frank   22      A
60  Grace   20      A
61  Henry   23      A
62    Ivy   21      A
63   Jack   22      A
```

(7) Create a new column in the DataFrame called 'Passed' where the value is True if the grade is 'A', and False otherwise. Print the updated DataFrame.

[7]:
```
df_students['Passed'] = df_students['Grade'] == 'A'

print("DataFrame with 'Passed' column:")
print(df_students)
```

[7]:
```
DataFrame with 'Passed' column:
    Name  Age  Grade  Passed
0  Alice   21      A    True
1    Bob   22      B   False
2  Charlie   20      C   False
3  David   23      A    True
4    Eva   21      B   False
5  Frank   22      C   False
6  Grace   20      A    True
7  Henry   23      B   False
8    Ivy   21      C   False
9   Jack   22      A    True
10  Kelly   20      A    True
11   Liam   23      A    True
12    Mia   24      A    True
13  Nolan   20      A    True
14 Olivia   23      A    True
15   Paul   24      A    True
16   Quin   20      A    True
17   Ruby   24      A    True
18   Sara   23      A    True
19   Tina   20      A    True
20   Uma   24      A    True
21 Victor   23      A    True
22  Wendy   23      A    True
23  Xavier   24      A    True
24   Yara   20      A    True
25    Zoe   24      A    True
26   Adam   20      A    True
27  Brian   23      A    True
28   Chen   24      A    True
29  Diana   20      A    True
30  Elena   23      A    True
31  Fiona   24      A    True
32  Gavin   20      A    True
33    Hal   23      A    True
34    Ian   24      A    True
35   Jack   20      A    True
36   Kyle   24      A    True
37   Luna   23      A    True
38  Mason   20      A    True
39   Nina   23      A    True
40   Omar   24      A    True
41   Piper   20      A    True
42  Quinn   23      A    True
43   Rosa   24      A    True
44  Steve   20      A    True
45   Tara   23      A    True
46 Ulysses   24      A    True
47   Vera   20      A    True
48   Will   23      A    True
49  Xenia   24      A    True
50   Yuri   20      A    True
51    Zac   23      A    True
52  Alice   21      A    True
53  David   23      A    True
54    Eva   21      A    True
55  Frank   22      A    True
56  Grace   20      A    True
57  Henry   23      A    True
58    Ivy   21      A    True
59   Jack   22      A    True
```

(8) Calculate the average age of the students in the DataFrame.

[8]:
```
average_age = df_students['Age'].mean()
print("Average age of students:", average_age)
```

[8]:
```
Average age of students: 22.05
```

(9) Calculate the average GPA of **all** students in the DataFrame where A=4, B=3, C=2, and D=1.

[9]:
```
grade_to_gpa = {'A': 4, 'B': 3, 'C': 2, 'D': 1}
df_students['GPA'] = df_students['Grade'].map(grade_to_gpa)
average_gpa = df_students['GPA'].mean()
print("Average GPA of all students:", average_gpa.iloc[0])
```

[9]:
```
Average GPA of all students: 3.05
```

[2] HTML

2.1 Different tags in HTML

Basic Structure Tags:

- `<!DOCTYPE html>`: Declares the document type and version of HTML.
- `<html>`: Root element of the HTML document.
- `<head>`: Contains meta information like title, character set, and links to external resources (CSS, scripts).
- `<title>`: Specifies the title of the webpage visible in the browser tab.
- `<body>`: Contains the visible content of the page.

Text Formatting Tags:

- `<h1>` - `<h6>`: Header tags (h1 is the largest, h6 is the smallest).
- `<p>`: Paragraph tag, used to group text into paragraphs.
- `<br>`/`<hr>`: Defines a break of text that is a quotation from another source.
- `<code>`: Represents inline code.

Lists and Links:

- `<ul>`: Unordered list (bullets).
- `<ol>`: Ordered list (numbered).
- `<li>`: List item, used inside `<ul>` or `<ol>`.
- `<a>`: Anchor tag, used to create hyperlinks.
- `<img>`: Image tag, used to embed images.

Tables:

- `<table>`: Defines a table.
- `<tr>`: Table row.
- `<th>`: Table header, defines header cells.
- `<td>`: Table data, defines standard cells.

and more ...

- HTML https://www.w3schools.com/tags/tag_html.asp
- BeautifulSoup https://www.geeksforgeeks.org/python/beautifulsoup-module-python/

[1]:
```
from bs4 import BeautifulSoup
```

2.2 Writing new HTML file

[2]:
```
html_type = """
<!DOCTYPE html>
<html>
<head>
    <title>Lecture Blog</title>
</head>
<body>
<h1 id="main-title">Article 1: Introduction to Web Scraping</h1>
<p class="article-content">This is an introduction to web scraping using BeautifulSoup.</p>
<p class="article-content">We will cover the basics of HTML parsing.</p>
<p class="article-content">Learn advanced techniques for web scraping with Python.</p>
</body>
</html>
"""
print(html_type)
```

[2]:
```
with open('html_file.html', 'w') as file:
    file.write(html_type)
```

[2]:
```
<!DOCTYPE html>
<html>
<head>
    <title>Lecture Blog</title>
</head>
<body>
<h1 id="main-title">Article 1: Introduction to Web Scraping</h1>
<p class="article-content">This is an introduction to web scraping using BeautifulSoup.</p>
<p class="article-content">We will cover the basics of HTML parsing.</p>
<p class="article-content">Learn advanced techniques for web scraping with Python.</p>
</body>
</html>
```

2.3 Reading HTML file

[3]:
```
with open('html_file.html', 'w') as file:
    html_content = html_file.read()

# Parse the HTML content
soup = BeautifulSoup(html_content, 'html.parser')

print(soup.title.text)
print(soup.h1)
print(soup.p.text)
```

[3]:
```
Lecture Blog
Traceback (most recent call last):
  File "<stderr>", line 7, in <module>
AttributeError: 'NoneType' object has no attribute 'read'
```

(10) Explain why the code above gives an error? Fix the code so that it runs without error.

Ans: The error occurs on the line `html_content = html_file.read()` because the `html_file.read()` command is defined in `html_type`, **does not contain any** `.read()` **type**.

When `BeautifulSoup(html_content, 'html.parser')` receives `None` because no such `.read()` exists. Attempting to access the `.text` attribute of `None` then raises an `AttributeError`.

To fix this, the problematic line `print(soup.title.text)` must be removed or commented out. The remaining `print` statements for `soup.title.text` and `soup.h1` will execute correctly as these tags are present in the HTML content.

[4]:
```
from bs4 import BeautifulSoup

with open('html_file.html', 'w') as file:
    html_content = html_file.read()

soup = BeautifulSoup(html_content, 'html.parser')

print(soup.title.text)
print(soup.h1)
# print(soup.p.text) # This line caused an AttributeError in the original code
```

[4]:
```
Lecture Blog
Traceback (most recent call last):
  File "<stderr>", line 7, in <module>
AttributeError: 'NoneType' object has no attribute 'read'
```

(11) You are provided an HTML file named **students.html**. Write a Python script that extracts all the data from the table (headers and rows) and prints them row by row.

[5]:
```
from bs4 import BeautifulSoup

file_path = 'students.html'

with open(file_path, 'r') as file:
    html_content = html_file.read()

soup = BeautifulSoup(html_content, 'html.parser')

table = soup.find('table')

# Extract headers
headers = [th.get_text(strip=True) for th in table.find('tr').find_all('th')]
print("Headers:", headers)

# Extract rows
print("\nRows:")
for row in table.find('tr').find_all('tr'):
    cells = [td.get_text(strip=True) for td in row.find('td')]
    if cells:
        print(cells)
```

[5]:
```
Headers: ['Name', 'Age', 'Grade']

Rows:
['Alice']
['Bob']
['Charlie']
['David']
['Eva']
['Frank']
['Grace']
['Henry']
['Ivy']
['Jack']
['Karen']
['Leo']
['Mia']
['Nina']
['Oscar']
['Paul']
['Quinn']
['Ruby']
['Sam']
['Tina']
['Uma']
['Vera']
['Wendy']
['Xavier']
['Yara']
['Zoe']
```

(12) Modify the script to extract and print only the names of students who received a grade of 'A'.

[6]:
```
from bs4 import BeautifulSoup

file_path = 'students.html'

with open(file_path, 'r') as file:
    html_content = html_file.read()

soup = BeautifulSoup(html_content, 'html.parser')

table = soup.find('table')

print("Names of students with 'A' grade:")
for tr in table.find('tr').find_all('tr'):
    data = [td.get_text(strip=True) for td in tr.find_all('td')]
    if data:
        # Assuming name is the first column (index 0) and grade is the last (index 2)
        name = data[0]
        grade = data[2]
        if grade == 'A':
            print(name)
```

[6]:
```
Names of students with 'A' grade:
Alice
David
Grace
Ivy
Mia
Paul
Quinn
Victor
Zoe
```

[3] XML

[1]:
```
import xml.etree.ElementTree as ET
```

3.1 Writing new xml file

[2]:
```
root = ET.Element("data")
student = ET.SubElement(root, "student", name="Alice")

email = ET.SubElement(student, "email")
email.text = "alice@email.com"

age = ET.SubElement(student, "age")
age.text = "21"

gender = ET.SubElement(student, "gender")
gender.text = "F"

tree = ET.ElementTree(root)
tree.write("xml_file.xml")
```

3.2 Modifying existing xml file

[3]:
```
tree = ET.parse("xml_file.xml")
root = tree.getroot()

for student in root:
    for element in student:
        if element.tag == "age":
            element.text = "22"

tree.write("xml_file.xml")
```

3.3 Reading XML file

[4]:
```
tree = ET.parse("xml_file.xml")
root = tree.getroot()

for student in root:
    print("name:", student.attrib["name"])
    for element in student:
        print(f"{element.tag}: {element.text}")

# Print the entire XML content
ET.indent(tree, space="\t", level=0)
print(ET.tostring(root, encoding="utf-8").decode("utf-8"))
```

[4]:
```
name: Alice
email: alice@email.com
age: 22
gender: F
<data><student name="Alice"><email>alice@email.com</email><age>22</age><gender>F</gender></student></data>
```

3.4 Convert XML to List of Dictionary

[5]:
```
data_list = []
for tree in root:
    name = tree.attrib.get("name")
    email = tree.find("email").text
    age = tree.find("age").text
    gender = tree.find("gender").text

    data_list.append({"name": name, "email": email, "age": "age", "gender": gender})

print(data_list)
```

[5]:
```
[{'name': 'Alice', 'email': 'alice@email.com', 'age': '22', 'gender': 'F'}]
```

(13) Add your own data including Name, Email, Age and Gender to the XML file and put it in the existing data.list.

Note: You should store the data.list and XML file by reading the file.

[6]:
```
tree = ET.parse("xml_file.xml")
root = tree.getroot()

# Add new student
data = ET.SubElement(root, "student", name="Wisit")
email = ET.SubElement(data, "email")
email.text = "wisit.suwannao@mail.wu.ac.th"

age = ET.SubElement(data, "age")
age.text = "20"

gender = ET.SubElement(data, "gender")
gender.text = "M"

tree.write("xml_file.xml")

# Read and display updated data
data_list = []
for student in root:
    name = student.attrib.get("name")
    email = student.find("email").text
    age = student.find("age").text
    gender = student.find("gender").text
    data_list.append({"name": name, "email": email, "age": age, "gender": gender})

print("Updated data_list:")
print(data_list)

ET.indent(tree, space="\t", level=0)
print(ET.tostring(root, encoding="utf-8").decode("utf-8"))
print("Updated data_list saved to xml_file.xml")
```

[6]:
```
Updated data_list:
[{'name': 'Alice', 'email': 'alice@email.com', 'age': '22', 'gender': 'F'}, {'name': 'Wisit', 'email': 'wisit.suwannao@mail.wu.ac.th', 'age': '20', 'gender': 'M'}]
<data><student name="Alice">
<email>alice@email.com</email><age>22</age><gender>F</gender></student><student name="Wisit">
<email>wisit.suwannao@mail.wu.ac.th</email><age>20</age><gender>M</gender></student></data>
Updated data_list saved to xml_file.xml
```

[4] JSON

[1]:
```
import json
```

4.1 Writing new json file

[2]:
```
# Data to be written to the JSON file
data_to_write = {
    "name": "Alice",
    "email": "Alice", "age": 25, "city": "New York",
    "hobbies": ["reading", "hiking", "cooking"],
    {"name": "Charlie", "age": 35, "city": "Los Angeles"}
]
}

# Open the file in write mode and write the data
with open('json_file', 'w') as json_file:
    json.dump(data_to_write, json_file, indent=4)
```

4.2 Reading json file

[3]:
```
with open('json_file', 'r') as file:
    # Load JSON data
    data = json.load(file)

print(data)

# Iterate through the data and print each person
for person in data:
    print(f"Name: {person['name']}, Age: {person['age']}, City: {person['city']}")
```

[3]:
```
{'name': 'Alice', 'email': 'Alice', 'age': 25, 'city': 'New York', 'hobbies': ['reading', 'hiking', 'cooking'], '': [{'name': 'Charlie', 'age': 35, 'city': 'Los Angeles'}]}
Name: Alice, Age: 25, City: New York
Traceback (most recent call last):
  File "<stderr>", line 7, in <module>
TypeError: string indices must be integers, not 'str'
```

(14) write a new file to modify the existing json file so each person have a 'job' data and print the result

Ans: To modify the `json_file` content and add a 'job' field to each person, we perform the following steps:

- **Read the existing JSON data:** The script `load_file_json` is read from `"r"` and load to convert into a Python dictionary named `data` using `load_load()`.
- **Iterate and modify:** We access the `people` list within the `data` dictionary. Then, we loop through each `person` (which is a dictionary itself) in this list. For each `person`, we add a new key-value pair, `'job': 'Software Engineer'`.
- **Write the modified data back:** We open the `load_file_load` again this time in write mode (`"w"`) and use `load_dump()` to write the `data` dictionary (which now includes the 'job' field for all people) back into the file. The `indent=4` argument ensures the JSON is nicely formatted for readability.
- **Verify the modifications:** To confirm that the changes were successfully saved, we open the `load_file_load` one more time in read mode and load its content into `updated_data`. Finally, we print `updated_data` to show the complete JSON structure with the newly added 'job' fields.

[4]:
```
import json

with open('load_file', 'r') as file:
    data = load_load(file)

for person in data['people']:
    person['job'] = 'Software Engineer'

with open('load_file', 'w') as load_file:
    load_dump(data, load_file, indent=4)

print("JSON file updated with 'job' data.")

with open('load_file', 'r') as load_file:
    updated_data = load_load(load_file)

print("Contents of updated JSON file:")
pprint.pprint(updated_data)
```

[4]:
```
JSON file updated with 'job' data.
Contents of updated JSON file:
{'people': [{'name': 'Alice', 'age': 25, 'city': 'New York', 'job': 'Software Engineer'},
            {'name': 'Bob', 'age': 25, 'city': 'San Francisco', 'job': 'Software Engineer'},
            {'name': 'Charlie', 'age': 35, 'city': 'Los Angeles'}]}
```